

# Programação II

## Aula 04



**UFMT**  
**Julho de 2025**

# Arrays

# Agenda

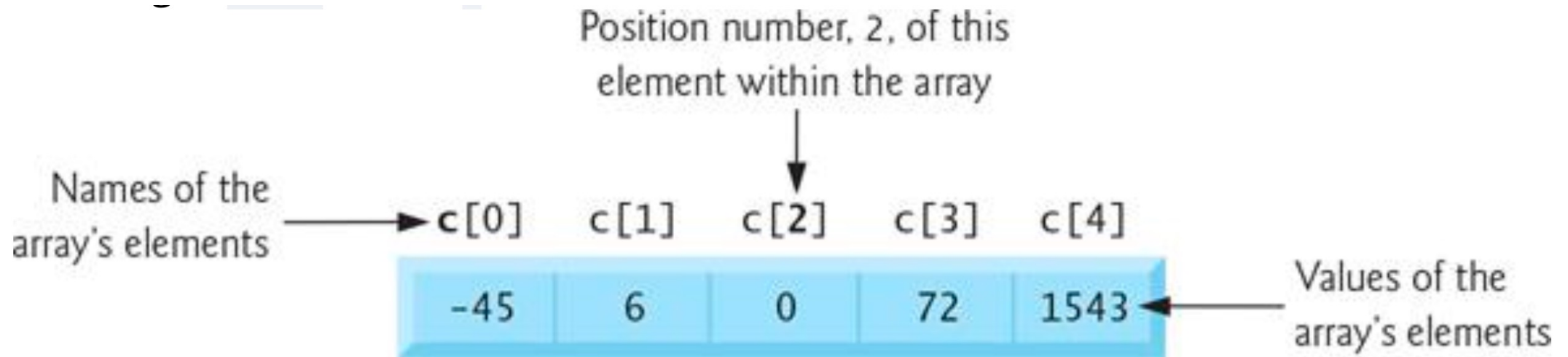
- Arrays de tamanho fixo estilo C.
- Arrays de tamanho fixo de C++ moderno (`std::array`).
- Arrays dinâmicos de C++ moderno (`std::vector`).

# Arrays Estilo C

# Introdução

- Os Arrays estilo C são estruturas de dados simples usadas para armazenar uma sequência de elementos de um mesmo tipo.
- Eles foram herdados da linguagem C e são integrados à linguagem C++. Isso significa que não precisamos incluir um arquivo de cabeçalho para usá-los.
- Eles têm tamanho fixo, conhecido em **tempo de compilação**, e **não armazenam** informação sobre seu próprio tamanho. Além disso, não fazem verificação de limites de índice (sem proteção contra acessos inválidos).
- Os elementos armazenados podem ser de qualquer tipo primitivo ou definido pelo usuário. Por exemplo, **int**, **float**, **char**, **string**, **bool**, etc.

# Indexação e Armazenamento



- Os elementos são indexados com valores inteiros começando com 0 usando o operador `[]`.
- O armazenamento é contíguo em memória.

# Declarando um Array Estilo C

- Para declarar de Array estilo C especificamos o tipos dos elementos e usamos colchetes ([ ]) para fornecer o comprimento do Array (um valor inteiro ou expressão constante). O comprimento do array deve ser pelo menos 1. O compilador emitirá um erro se o comprimento for zero, negativo ou um valor não inteiro.

```
1  int main()  
2  {  
3      //declara e inicializa um array no estilo C com 5 elementos  
4      //do tipo int, todos zerados.  
5      int teste[5] {};  
6  
7      return 0;  
8  }
```

# Inicializando um Array Estilo C

- Um array estilo C pode ser inicializado com valores diferentes do default do tipo diretamente em sua declaração.
- O comprimento pode ser omitido se o compilador puder inferir o número de elementos dos valores de inicialização.

```
1  #include <string>
2  int main()
3  {
4      //declara e inicializa um array de inteiros e um array de strings
5      int arr1[5] {1, 2, 3, 4, 5};
6      std::string arr2[5] {"linder", "ufmt", "prog2"};
7
8      return 0;
9  }
```



# Acessando os Elementos de um Array Estilo C

- Para acessar os elementos de um Array estilo C use o operador []. Cuidado! O operador [] não faz nenhuma verificação de limites. Isso significa que passar um índice fora dos limites resultará em comportamento indefinido.

```
1  #include <iostream>
2  int main()
3  {
4      //declara e inicializa
5      int arr[5] {1, 2, 3, 4, 5};
6
7      //acessa valores
8      std::cout << arr[0] << " " << arr[4];
9      return 0;
10 }
```

# Array Estilo C: Armazenamento

- Os elementos de um Array estilo C são armazenados em posições consecutivas da memória. O tamanho do array (em bytes) pode ser obtido com o operador `sizeof()`.

```
1  #include <iostream>
2  int main()
3  {
4      //declara e inicializa
5      int arr[5] {1, 2, 3, 4, 5};
6
7      //acessa valores dos endereços
8      std::cout << &arr[0] << " " << &arr[1] << " " << &arr[2] << std::endl;
9      std::cout << sizeof(arr);
10
11     return 0;
12 }
```

# Array Estilo C Constantes

- Assim como qualquer outra variável, Arrays podem ser especificados com o modificador **const**. Neste caso, os valores dos elementos não podem ser modificados.

```
1  int main()  
2  {  
3      const int prime[5] { 2, 3, 5, 7, 11 }; // um array de inteiros constantes  
4      prime[0] = 17; // erro de compilação  
5  
6      return 0;  
7  }
```

# Obtendo o Comprimento de um Array Estilo C

- A partir do C++17, podemos usar `std::size()`, definido em `<iterator>`, para obter o comprimento do Array como um valor inteiro sem sinal.
- No C++20, também podemos usar `std::ssize()`, que retorna o comprimento do Array como um valor inteiro com sinal.

```
1  #include <iostream>
2  int main()
3  {
4      int prime[5] { 2, 3, 5, 7, 11 }; // um array de inteiros constantes
5
6      std::cout << std::size(prime) << '\n'; // C++17, unsigned int = 5
7      std::cout << std::ssize(prime) << '\n'; // C++20, signed int = 5
8
9      return 0;
10 }
```

# Atribuição de Valores

- Surpreendentemente, os arrays estilo C de C++ não suportam atribuição de um conjunto de valores. Os elementos devem ser atualizados individualmente.

```
1  int main()  
2  {  
3      int arr[] { 1, 2, 3 }; // ok!  
4      arr[0] = 4;           // ok!  
5      arr = { 5, 6, 7 };    // Erro!  
6  
7      return 0;  
8  }
```

# Laço **for** Simplificado

- Também chamado de range-based **for**, é uma forma simplificada de percorrer elementos de containers, tais como arrays em C++. Esse recurso introduzido no C++11.
- Sintaxe Básica:

```
1  for (tipo_variavel nome : container) {  
2  |    // corpo do loop  
3  }
```

# Laço **for** Simplificado

- No exemplo abaixo, a cada iteração do laço, a variável **valor** assume um dos elementos do Array **arr**. Primeiro 1, depois 2, depois 3, e assim sucessivamente.

```
1  #include <iostream>
2  int main()
3  {
4      int arr[5] {1, 2, 3, 4, 5};
5
6      for (int valor : arr) {
7          std::cout << valor << std::endl;
8      }
9
10     return 0;
11 }
```

# Arrays Estilo C Multidimensionais

- Arrays multidimensionais são armazenados consecutivamente na memória, em **ordem de linhas** (*row-major order*). A seguir especificamos uma matriz (Array 2D), onde {10, 20, 30} corresponde à primeira linha; {40, 50, 60} corresponde à segunda linha e {70, 80, 90} corresponde à terceira linha.

```
1  #include <iostream>
2  int main() {
3      int a[3][3] = {{10, 20, 30}, {40, 50, 60}, {70, 80, 90}};
4
5      for (int i = 0; i < 3; ++i){
6          for (int j = 0; j < 3; ++j){
7              std::cout << a[i][j] << " ";
8          }
9      }
10
11     return 0;
12 }
```



# Arrays Estilo C Multidimensionais

- Percorrendo um Array multidimensional 2D com um laço **for** simplificado.

```
1  #include <iostream>
2  int main() {
3      int a[3][3] = {{10, 20, 30}, {40, 50, 60}, {70, 80, 90}};
4
5      for (int (&linha)[3] : a) {           // 'linha' é um array de 3 ints
6          for (int x : linha) {             // percorre os elementos da linha
7              std::cout << x << " ";
8          }
9          std::cout << "\n";
10     }
11
12     return 0;
13 }
```

Poderia se  
substituído por  
auto& linha



# Passagem de Array Estilo C para Função

- Quando um Array estilo C é passado para uma função, o que de fato é passado é o endereço do primeiro elemento do Array — não o Array inteiro.
- A função pode usar o endereço base para acessar os elementos, mas não tem “consciência” de que se trata de um container.
- Portanto, isso impede o uso do laço **for** simplificado ou de funções como `std::size()`

```
1  #include <iostream>
2
3  void imprime(int p[]) {
4      std::cout << p[0] << std::endl;
5  }
6
7  int main() {
8      int a[] = {10, 20, 30};
9      int b[] = {25, 35, 45, 55};
10     imprime(a); // 'a' decai para 'int*'
11     imprime(b); // 'a' decai para 'int*'
12
13     return 0;
14 }
```

# Passagem de Array Estilo C para Função

- A forma que Arrays são passados para funções possibilita que a função receba Arrays de qualquer tamanho.
- Entretanto, o Array não carrega seu tamanho e, por isso, a solução mais adotada consiste em passar o tamanho do Array como um segundo parâmetro.

```
1  #include <iostream>
2
3  void printArray(int arr[], int tam){
4      for(int i{0}; i < tam; i++){
5          std::cout << arr[i] << std::endl;
6      }
7  }
8
9  int main()
10 {
11     int arr[5] {1, 2, 3, 4, 5};
12
13     printArray(arr, 5);
14
15     return 0;
16 }
```

# Passagem de Array Estilo C para Função

- Passar um array multidimensional estilo C para uma função C++ exige que você especifique todas as dimensões exceto a primeira.
- Isso é necessário porque o compilador precisa saber como calcular o deslocamento correto na memória para acessar os elementos.

```
1  #include <iostream>
2
3  void imprime(int m[][3], int linhas) {
4      for (int i = 0; i < linhas; ++i)
5          for (int j = 0; j < 3; ++j)
6              std::cout << m[i][j] << " ";
7  }
8
9  int main() {
10     int a[3][3] = {{10, 20, 30}, {40, 50, 60}, {70, 80, 90}};
11
12     imprime(a, 3);
13
14     return 0;
15 }
```

# Passagem de Array Estilo C para Função

- Embora o decaimento do array tenha sido uma solução inteligente para garantir que Arrays estilo C de diferentes comprimentos pudessem ser passados para uma função sem fazer cópias caras, a perda da informação de comprimento do array facilita a ocorrência de vários tipos de erros.
- Historicamente, os programadores contornam a falta de informações sobre o comprimento do array passando-o como um segundo argumento. Vimos esse artifício no exemplo anterior.

# Arrays de Tamanho Fixo de C++11

# Declarando Arrays C++11

- Para criar um objeto Array inclua o arquivo `<array>` e obedeça à seguinte regra de declaração :

`std::array<tipo, tamanho> nome;`

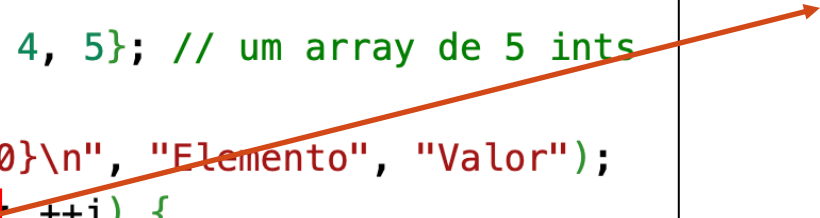
- A notação `<tipo, tamanho>` indica que `array` é um template de classe. Ou seja, o compilador pode criar objetos `array` de diferentes tipos, como de `int`, `double`, `string`, etc.
- Diferente dos Arrays estilo C, os Arrays de C++ são objetos que guardam internamente a informação sobre seu tamanho, que pode ser recuperada invocando o método `size()` do objeto.

# Declarando e Acessando Arrays C++11

- Na inicialização, se houver menos valores do que elementos do array, os elementos restantes são inicializado com um valor padrão.

```
1  #include <format>
2  #include <iostream>
3  #include <array>
4
5  int main()
6  {
7      std::array<int, 5> values {1, 2, 3, 4, 5}; // um array de 5 ints
8
9      std::cout << std::format("{:>7}{:>10}\n", "Elemento", "Valor");
10     for (size_t i{0}; i < values.size(); ++i) {
11         std::cout << std::format("{:>7}{:>10}\n", i, values[i]);
12     }
13
14     return 0;
15 }
```

Recupera o  
tamanho do array





# Acesso e Verificação de Limites para Índices de Array

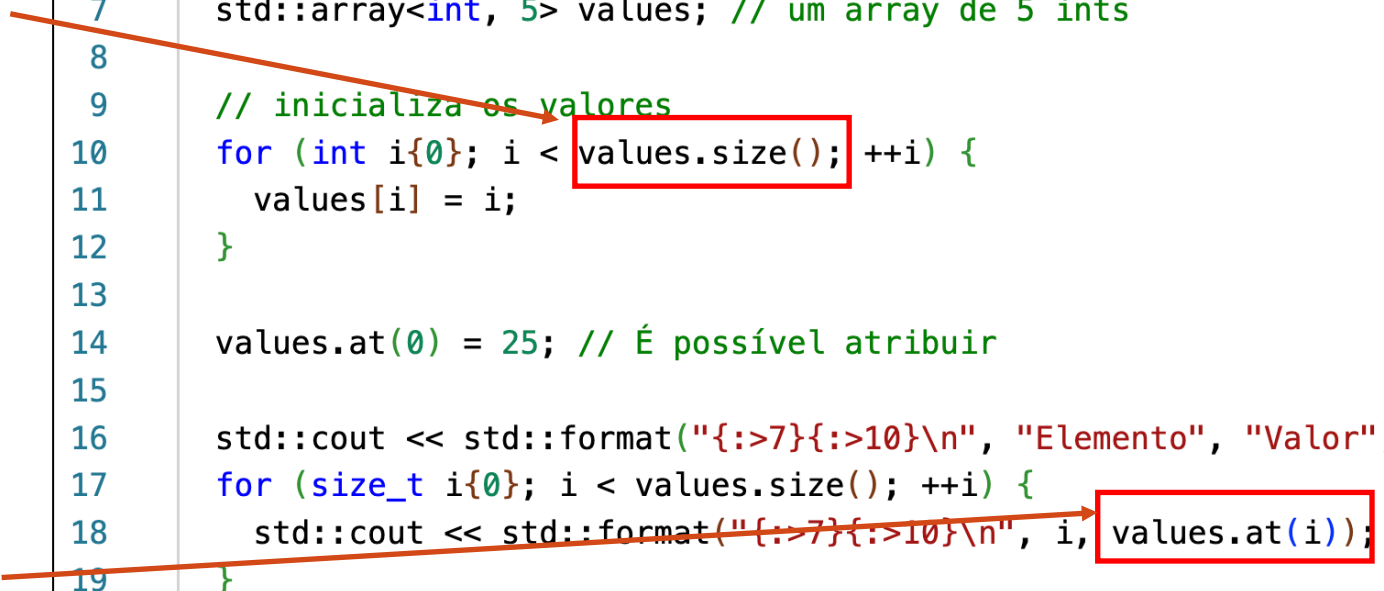
- Para acessar os elementos de uma Array podemos usar o operador `[ ]`, tal como em Arrays estilo C. Entretanto, esse método não fornece verificação automática dos limites. Assim, um programa em execução pode acessar além da extremidade de um Array sem que o erro seja notificado.
- Um método de acesso alternativo, consiste em usara função membro `at()` da classe Array. O método `at()` verifica os limites do Array antes de acessar seus elementos. Se a função `at()` encontrar um índice fora dos limites, ela gera um erro em tempo de execução conhecido como **exceção**.
- Veremos como tratar exceções em aulas futuras.

# Verificação de Limites para Índices de Array

Em um loop que processa elementos de um array, certifique-se de que a condição de término impeça o acesso a elementos fora dos limites da array.

A função membro `at()` realiza a verificação de limites antes de acessar o elemento.

```
1  #include <format>
2  #include <iostream>
3  #include <array>
4
5  int main()
6  {
7      std::array<int, 5> values; // um array de 5 ints
8
9      // inicializa os valores
10     for (int i{0}; i < values.size(); ++i) {
11         values[i] = i;
12     }
13
14     values.at(0) = 25; // É possível atribuir
15
16     std::cout << std::format("{:>7}{:>10}\n", "Elemento", "Valor");
17     for (size_t i{0}; i < values.size(); ++i) {
18         std::cout << std::format("{:>7}{:>10}\n", i, values.at(i));
19     }
20
21     return 0;
22 }
```



# Percorrendo Arrays com **for** Simplificado

- Em geral, se for processar todos os elementos de um Array, use o laço **for** simplificado, pois ela garante que seu código não sairá dos limites do Array.

Use referência para evitar cópia.  
Use const se não for modificar o  
valor

```
1  #include <format>
2  #include <iostream>
3  #include <array>
4
5  int main()
6  {
7      std::array<int, 5> valores {1, 2, 3, 4, 5}; // um array de 5 ints
8
9      std::cout << std::format("{:>7}{:>10}\n", "Valor", "Quadrado");
10     for (const int& valor: valores) {
11         std::cout << std::format("{:>7}{:>10}\n", valor, valor*valor);
12     }
13
14     return 0;
15 }
```

# Uma Introdução à `constexpr`

- O C++11 introduziu o qualificador `constexpr` para declarar variáveis que podem ser avaliadas em tempo de compilação e resultar em uma constante.
- Uma variável `constexpr` deve ser inicializada em tempo de compilação, diferente de uma variável `const` pode ser inicializada em tempo de execução.
- No exemplo a seguir usamos `constexpr` para definir o tamanho de um array.

# Exemplo

Define uma constante  
inteira

```
1  #include <format>
2  #include <iostream>
3  #include <array>
4
5  int main() {
6      // usado para especificar o tamanho do array
7      constexpr size_t arraySize{5};
8
9      std::array<int, arraySize> values{};
10
11     // atribui valores
12     for (int i{0}; i < values.size(); ++i) {
13         values.at(i) = 2 + 2 * i;
14     }
15
16     // mostra
17     for (const int& value : values) {
18         std::cout << std::format("{} ", value);
19     }
20
21     return 0;
22 }
```

Retorna uma referência  
ao elemento do Array.  
Portanto pode receber  
valores.

# Ordenação de Arrays

- Para ordenar um Array em ordem ascendente usamos a a função **sort** definida em `<algorithm>`.
- O padrão C++ indica que **sort** usa um algoritmo  $O(n \log n)$ , mas não especifica qual.
- **`std::sort()`** pode ser usado com **arrays** de tipos primitivos e objetos C++, desde que os objetos **tenham o operador `<` definido**.
- Veja um exemplo no slide a seguir.

# Exemplo

Ordena em ordem  
ascendente

```
1  #include <array>
2  #include <algorithm> // contains sort and binary_search
3  #include <format>
4  #include <iostream>
5  #include <string>
6
7  int main() {
8
9      std::array<std::string, 7> colors{"red", "orange", "yellow", "green", "blue", "indigo", "violet"};
10
11     std::cout << "Array desordenado:\n ";
12     for (const std::string& color : colors) {
13         std::cout << std::format("{} ", color);
14     }
15
16     // ordena
17     std::sort(std::begin(colors), std::end(colors));
18
19     std::cout << "\nArray ordenado:\n ";
20     for (const std::string& color : colors) {
21         std::cout << std::format("{} ", color);
22     }
23
24     return 0;
25 }
```

# Busca em Arrays

- A consulta pode ser feita usando a função `std::binary_search`.

```
1  #include <array>
2  #include <algorithm> // contains sort and binary_search
3  #include <format>
4  #include <iostream>
5  #include <string>
6
7  int main() {
8
9      std::array<std::string, 7> colors{"red", "orange", "yellow", "green", "blue", "indigo", "violet"};
10
11      // ordena
12      std::sort(std::begin(colors), std::end(colors));
13
14      // busca por "indigo"
15      bool found{std::binary_search(std::begin(colors), std::end(colors), "indigo")};
16      std::cout << std::format("\n\n\"indigo\" {} encontrado no array\n", found ? "foi" : "nao foi");
17
18      // busca por "cyan"
19      found = std::binary_search(std::begin(colors), std::end(colors), "cyan");
20      std::cout << std::format("\n\n\"cyan\" {} encontrado no array\n", found ? "foi" : "nao foi");
21      return 0;
22 }
```



# Passagem de Arrays para Funções

- A passagem de um array para uma função é mais simples que no caso de array estilo C, pois o objeto incorpora a informação sobre o tamanho do array.

```
1  #include <array>
2  #include <iostream>
3  #include <string>
4
5  constexpr int tamanho{7};
6  void printArray(std::array<std::string, tamanho>& arr){
7      for(auto& elemento: arr){
8          std::cout << elemento << std::endl;
9      }
10 }
11
12 int main() {
13
14     std::array<std::string, 7> cores{"red", "orange", "yellow", "green", "blue", "indigo", "violet"};
15
16     printArray(cores);
17
18     return 0;
19 }
```

# Arrays Multidimensionais

- Você pode usar Arrays com duas dimensões para representar tabelas de valores com dados organizados em linhas e colunas.
- Para identificar um elemento específico da tabela, precisamos especificar dois índices.

The diagram illustrates a 2D array structure. It features a grid of 3 rows and 4 columns. The columns are labeled 'Column 0', 'Column 1', 'Column 2', and 'Column 3' at the top. The rows are labeled 'Row 0', 'Row 1', and 'Row 2' on the left. Each cell in the grid contains a subscripted array element, such as 'a[0][0]' in the top-left cell. Below the grid, three arrows point to the components of the element 'a[2][1]': the leftmost arrow points to 'a' and is labeled 'array name'; the middle arrow points to '[2]' and is labeled 'Row subscript'; the rightmost arrow points to '[1]' and is labeled 'Column subscript'.

|       | Column 0 | Column 1 | Column 2 | Column 3 |
|-------|----------|----------|----------|----------|
| Row 0 | a[0][0]  | a[0][1]  | a[0][2]  | a[0][3]  |
| Row 1 | a[1][0]  | a[1][1]  | a[1][2]  | a[1][3]  |
| Row 2 | a[2][0]  | a[2][1]  | a[2][2]  | a[2][3]  |

array name  
Row subscript  
Column subscript

# Array Bidimensional: Exemplo 1

```
1  #include <array>
2  #include <iostream>
3  #include <string>
4
5  int main() {
6
7      std::array<std::array<int, 3>, 3> matriz = {std::array{1,2,3}, std::array{4,5,6}, std::array{7,8,9}};
8
9      // loop linha por linha
10     for (const auto& linha : matriz) {
11         for (const auto& elemento : linha) {
12             std::cout << elemento << ' ';
13         }
14         std::cout << std::endl;
15     }
16
17     return 0;
18 }
```

# Array Bidimensional: Exemplo 2

```
1  #include <array>
2  #include <iostream>
3  #include <string>
4
5  int main() {
6
7      std::array<std::array<int, 3>, 3> matriz = {std::array{1,2,3}, std::array{4,5,6}, std::array{7,8,9}};
8
9      // loop linha por linha
10     for (int i{0}; i < matriz.size(); i++) {
11         for (int j{0}; j<matriz.at(i).size(); j++) {
12             std::cout << matriz.at(i).at(j)<< ' ';
13         }
14         std::cout << std::endl;
15     }
16
17     return 0;
18 }
```

# Vectors

# Criando Objetos `vector`

- Um `vector` (vetor) é um template de classe semelhante a um `Array`, mas suporta redimensionamento dinâmico. Por default, os elementos são inicializados para 0.

```
1  #include <iostream>
2  int main()
3  {
4      //declara e inicializa
5      std::vector<int> inteiros(7); // vector de 7 elementos int
6      std::vector<int> inteiros(10); // vector de 10 elementos int
7
8      return 0;
9  }
```

# Criando Objetos `vector`

- Vectors podem ser inicializados usando lista de inicialização.

```
1  #include <iostream>
2  int main()
3  {
4      //declara e inicializa
5      std::vector<int> inteiros{1, 2, 3, 4};
6
7      return 0;
8  }
```

# Função Membro `size()`

- O número de elementos de um vector pode ser obtido com a função membro `size()`

```
1  #include <iostream>
2  #include <vector>
3  int main()
4  {
5      //declara e inicializa
6      std::vector<int> inteiros{1, 2, 3, 4};
7
8      std::cout << inteiros.size();
9
10     return 0;
11 }
```



# Passando Vectors Para Funções

- Vectors podem ser grandes e, por isso, em geral devem ser passados por referência.
- O exemplo abaixo inicializa um vector com valores fornecidos pelo usuário. A implementação usa um **for** simplificado, mas outros laços também podem ser empregados, uma vez que vectors podem ser indexados com `[]` ou método `at()`.

```
// input vector contents
void inputVector(std::vector<int>& items) {
    for (int& item : items) {
        std::cin >> item;
    }
}
```

# Acessando e Modificando Elementos de Vectors

- Use a função membro **at()** para obter um elemento de um vector ou para atribuir um valor para um elemento.
- Se o índice for válido, a função membro **at()** retorna:
  - uma referência ao elemento naquele local, que pode ser usada para alterar o valor do elemento correspondente, ou
  - uma referência constante ao elemento naquele local, que não pode ser usada para alterar o valor do elemento correspondente, mas pode ser usada para ler o valor do elemento.
- Se **at()** for chamado em um vector constante ou por meio de uma referência declarada como **const**, a função retorna uma referência constante.

# Comparando Vectors

- A biblioteca padrão C++ sobrecarrega os operadores `==` e `!=` para comparar Arrays e vectors quanto a igualdade.
- `==` retorna true se todos os elementos forem iguais.
- `!=` retorna true se pelo menos um elemento for diferente.

```
1  #include <iostream>
2  #include <vector>
3
4  int main()
5  {
6      //declara e inicializa
7      std::vector<int> inteiros1{1, 2, 3, 4};
8      std::vector<int> inteiros2(10);
9
10     if(inteiros1 == inteiros2){
11         std::cout << "Sao iguais!!!" << std::endl;
12     }
13
14     return 0;
15 }
```

# Inicialização com outro Vector

Um vector pode ser inicializado copiando o conteúdo de um vector existente.

```
1  #include <iostream>
2  #include <vector>
3
4  int main()
5  {
6      //declara e inicializa
7      std::vector<int> inteiros1{1, 2, 3, 4};
8      std::vector<int> inteiros2 {inteiros1}; //construtor de cópia
9
10     if(inteiros1 == inteiros2){
11         std::cout << "Sao iguais!!!" << std::endl;
12     }
13
14     return 0;
15 }
```

# Atribuição de Vectors

Um vector pode ser atribuído a outro vector. A atribuição gera uma cópia do conteúdo.

```
1  #include <iostream>
2  #include <vector>
3
4  int main()
5  {
6      //declara e inicializa
7      std::vector<int> inteiros1{1, 2, 3, 4};
8      std::vector<int> inteiros2 = inteiros1;
9
10     if(inteiros1 == inteiros2){
11         std::cout << "Sao iguais!!!" << std::endl;
12     }
13
14     return 0;
15 }
```

# Alterando o Tamanho de um Vetor

- Uma diferença fundamental entre um vector e um Array é que um vector pode crescer e encolher dinamicamente conforme chama o método `push_back()`.
- `push_back()`, se necessário, aloca memória e insere um elemento no final do vector.

```
1  #include <iostream>
2  #include <vector>
3  int main()
4  {
5      std::vector<int> vec {1, 2, 3, 4, 5};
6      std::cout << "\ntamanho atual: " << vec.size() <<std::endl; // imprime 5
7
8      vec.push_back(1000);
9      std::cout << "\ntamanho apos push_back(): " << vec.size() <<std::endl; //imprime 6
10
11     return 0;
12 }
```

# Alterando o Tamanho de um Vetor

- **back()** recupera o elemento que está no final do vector e **pop\_back()** remove o elemento que está no final do vector.

```
1  #include <iostream>
2  #include <vector>
3
4  int main() {
5      std::vector<int> v = {10, 20, 30};
6
7      int ultimo = v.back(); // lê o último (30)
8      v.pop_back();          // remove o último
9
10     std::cout << "Removido: " << ultimo << "\n";
11
12     return 0;
13 }
```